LAB 13.2

Code Refactoring – Improving Legacy Code with AI Suggestions

N.Vishnu

2303A51210
B.Tech CSE

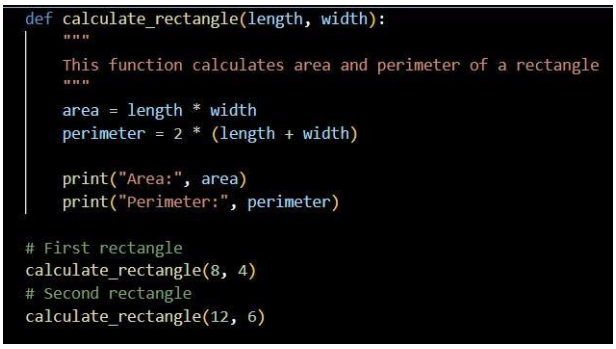
AI Assisted Coding
23CS002PC304
III / II

Task 1: Refactoring – Eliminating Repetitive Logic

PROMPT

Identify repeated computations in the given Python program and refactor the code by creating reusable functions. Ensure that the output remains the same and add proper docstrings.

CODE



```
def calculate_rectangle(length, width):
    """
    This function calculates area and perimeter of a rectangle
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)

# First rectangle
calculate_rectangle(8, 4)
# Second rectangle
calculate_rectangle(12, 6)
```

def calculate_rectangle(length, width):

“”This function calculates area and perimeter of a rectangle”

area = length * width perimeter = 2 * (length + width)

print("Area:", area) print("Perimeter:", perimeter)

 # First rectangle calculate_rectangle(8,

4)

 # Second rectangle

calculate_rectangle(12, 6)

OUTPUT

```
PS D:\3-2 SEM\AI ASSISTED>
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

RESULT

The program was successfully refactored by replacing repeated calculations with a reusable function.

EXPLANATION

The original program repeated the same logic multiple times for calculating area and perimeter.
To improve maintainability, a function named calculate_rectangle() was created.

Benefits:

- **Reduces code repetition**
- **Improves readability**
- **Makes the program reusable and maintainable**

Task 2: Refactoring – Modularizing Inline Calculations

PROMPT

Identify repeated inline calculations in the Python program and convert them into reusable functions with proper documentation.

CODE

```
def calculate_final_amount(amount):
    """
    Calculates final amount after applying 10% discount
    """
    discount = amount * 0.10
    final_amount = amount - discount
    print("Final Amount:", final_amount)


calculate_final_amount(1000)
calculate_final_amount(2000)
```

def calculate_final_amount(amount):

"""

Calculates final amount after applying 10% discount

""" discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

calculate_final_amount(1000)

calculate_final_amount(2000)

OUTPUT

```
PS D:\3-2 SEM\AI ASSISTED>
Final Amount: 900.0
Final Amount: 1800.0
```

RESULT

The repeated discount calculation logic was modularized using a reusable function.

EXPLANATION

The original code repeated discount calculations for different amounts. The logic was moved into a function calculate_final_amount().

Advantages:

- **Improves modularity**
- **Makes the code reusable**
- **Reduces duplication**

Task 3: Refactoring – Improving Loop and Conditional Performance

PROMPT

Optimize the given Python code by replacing inefficient nested loops with faster lookup techniques while maintaining the same program output.

CODE

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

student_set = set(students)

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
```

OUTPUT

```
PS D:\3-2 SEM\AI ASSISTED>
Charan found
Esha not found
Bhavya found
```

RESULT

The nested loop was optimized using a set data structure for faster lookup.

EXPLANATION

The original program used nested loops which increased time complexity.

Old Complexity O(n × m)

New Complexity O(n)

Using a set improves lookup performance because sets support constant time membership checking.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

PROMPT

Detect hardcoded numeric values in the program and replace them with named constants to improve readability and maintainability

CODE

```
RATE = 2
TIME = 5

def calculate_simple_interest(principal):
    interest = (principal * RATE * TIME) / 100
    print("Simple Interest:", interest)


calculate_simple_interest(1000)
calculate_simple_interest(2000)
```

OUTPUT

```
PS D:\3-2 SEM\AI ASSISTED>
Simple Interest: 100.0
Simple Interest: 200.0
```

RESULT

Hardcoded numeric values were replaced with constants

EXPLANATION

Instead of directly writing numbers in formulas, constants RATE and TIME were created.

Advantages:

- **Improves readability**
- **Makes future updates easier**
- **Follows Python coding standards**

Task 5: Refactoring – Enhancing Variable Naming and Code Clarity

PROMPT

Improve the readability of the program by renaming unclear variables and adding meaningful comments without changing the program output.

CODE

```
# Define two numbers
num1 = 50
num2 = 30

# Calculate their sum
total_sum = num1 + num2

# Display result
print(total_sum)
```

 # Define two numbers

num1 = 50 num2 = 30

 # Calculate their sum total_sum

= num1 + num2

 # Display result print(total_sum)

OUTPUT

```
PS D:\3-2 SEM\AI
80
```

RESULT

Variable names were improved and comments were added to enhance readability.

EXPLANATION

The original variables **x, y, z** were not descriptive. They were renamed to:

- **num1 □ num2**
- **total_sum**

Comments were also added to explain the logic, making the code easier to understand and maintain.